

Sesión 11: Modelado Base y Arquitecturas de Despliegue

Evolución de Sistemas Distribuidos, Fundamentos del Modelo Cliente-Servidor y Diseño Físico Multi-Capas (N-Tier)

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

| Agenda del Día (1/2)

 Introducción a Sistemas Distribuidos y Web

 El Paradigma Cliente-Servidor: Roles y Flujo

 Arquitectura Física Multi-Capas (N-Tier)

 Anatomía de una Petición: IP, Puertos y DNS



El Salto a la Web

Dejamos atrás las aplicaciones aisladas de consola para dominar los ecosistemas globales conectados por redes de alta velocidad.

Agenda del Día (2/2)



Lógica de Negocio

Por qué separar el procesamiento de datos de la interfaz visual del usuario.



Capa de Datos

Estrategias básicas de persistencia y almacenamiento centralizado.



Solidaridad

Valor de la unidad: Desarrollar software accesible que potencie y colabore con la comunidad.



Evolución de las Arquitecturas de Software

De sistemas monolíticos aislados a infraestructuras web globales distribuidas

La Limitación de las Aplicaciones de Escritorio

En el software tradicional (como el que veníamos construyendo en consolas), todo el código (interfaz, lógica y almacenamiento) reside en la misma máquina física.

El Problema del Monolito Local: Si los datos cambian o se actualizan, otra computadora en otra ubicación geográfica no tiene forma de enterarse. No hay sincronización centralizada, escalabilidad ni concurrencia real.



Máquina Local Aislada

[UI] + [Lógica] + [Datos Locales]

La Solución: El Ecosistema Web

La web transforma radicalmente el paradigma. La computación se descentraliza mediante nodos de red especializados que cooperan entre sí para cumplir una tarea.

Definición de Sistema Distribuido:

Colección de computadoras independientes que se presentan ante los usuarios finales como un único sistema coherente y unificado a través de internet.



El Modelo Cliente-Servidor

El pilar fundamental de comunicación interactiva en redes de datos

Anatomía del Modelo Cliente-Servidor

Este modelo divide los nodos de la red en dos roles asimétricos bien definidos:

El Cliente (Client)

Es el iniciador de la comunicación. Dispositivo que solicita un servicio o recurso (ej. Navegadores web como Chrome, Apps móviles, consolas de comandos remotas).

El Servidor (Server)

Nodo que permanece en escucha pasiva constante. Recibe las solicitudes, procesa la lógica requerida, valida accesos y retorna una respuesta estructurada.

El Ciclo de Petición – Respuesta (Request-Response)

Toda la interacción en la web se rige por un flujo transaccional estrictamente síncrono o asíncrono basado en dos pasos esenciales:

1. HTTP Request (Petición): El cliente empaqueta datos (parámetros, URL, encabezados) y los envía al servidor indicando qué acción desea realizar.

2. HTTP Response (Respuesta): El servidor procesa la petición y responde con un código de estado (ej. 200 OK, 404 Not Found) y el cuerpo de la información solicitada.



Arquitectura Física en Capas (N-Tier)

Estrategias de desacoplamiento y distribución física de componentes de software

| Capas Lógicas vs Niveles Físicos (Layers vs Tiers)

Antes de diseñar, un ingeniero debe diferenciar con claridad estos dos conceptos frecuentemente confundidos:

- **Capas Lógicas (Layers):** División conceptual del código dentro del mismo proyecto (ej. Clases de datos independientes de las clases visuales).
- **Niveles Físicos (Tiers):** Distribución real del software en ****computadoras o servidores físicamente separados**** a través de la red.

Estudiaremos el modelo estándar de la industria: La Arquitectura clásica de 3 Niveles Físicos.

Nivel 1: Capa de Presentación (Presentation Tier)

Es la capa más externa del sistema, encargada de la interfaz gráfica y la interacción directa con el usuario final.

Tecnologías Comunes:

HTML5, CSS3, JavaScript puro, frameworks modernos (Angular, React) o vistas generadas dinámicamente desde el servidor.

Responsabilidad Única: Capturar las acciones del usuario, enviarlas al siguiente nivel y renderizar los datos devueltos de forma atractiva y ordenada. **No realiza cálculos complejos ni accede a la base de datos**.

Nivel 2: Capa de Aplicación o Negocio (Application Tier)

El "cerebro" del sistema. Aquí se procesan todas las reglas del negocio, validaciones de seguridad, algoritmos de cálculo y orquestación del flujo de datos.

</> Tecnologías Comunes:

Servicios backend en **C# (.NET Core)**, Java (Spring Boot), Python, Node.js.

Aislamiento Estratégico: Al estar aislada en su propio servidor, si la interfaz de usuario cambia por completo, la lógica matemática permanece intacta y funcional, protegiendo la inversión de código.

Nivel 3: Capa de Datos (Data Tier)

El nivel responsable del almacenamiento persistente y seguro de la información del sistema.

Motores de Almacenamiento:

Sistemas relacionales (SQL Server, PostgreSQL, MySQL) y estructuras NoSQL.

Seguridad y Control: Solo el Nivel de Aplicación tiene permitido consultar este nivel. Ningún usuario o cliente externo puede lanzar comandos directos a la base de datos, previniendo ataques de inyección destructivos.

Fundamentos de Redes para Desarrolladores Web

Cómo viajan los bits e identificadores físicos a través del cableado global de internet

Direcciones IP: Identidad en la Red

Una dirección IP (Internet Protocol) es un identificador numérico único asignado a cada dispositivo conectado a una red informática.

IPv4: 192.168.1.25

Dirección Local (Loopback): 127.0.0.1

Nombre Clave Local: localhost

Permite ubicar geográficamente y rutar los paquetes de datos desde la computadora de origen hasta el servidor de destino exacto en cualquier parte del mundo.

Puertos lógicos: Direccionamiento interno

Si la dirección IP es el equivalente al número de un edificio de apartamentos, el ****puerto lógico**** representa el número de apartamento específico.

Permite que una sola computadora ejecute múltiples servicios de red simultáneamente sin que interfieran entre sí:

Puerto 80

Tráfico Web Estándar No Seguro (HTTP)

Puerto 443

Tráfico Web Encriptado Seguro (HTTPS)

Puerto 5000+

Puertos de desarrollo local comunes en .NET

DNS: El traductor de internet

Los seres humanos memorizan nombres, las computadoras procesan números. El ****DNS (Domain Name System)**** resuelve esta brecha.

"www.google.com"

142.250.190.46

Es una base de datos distribuida globalmente encargada de traducir cadenas de texto legibles (Dominios) en las direcciones IP numéricas de los servidores correspondientes.

Protocolos de Comunicación Web: HTTP

Reglas y especificaciones formales para el intercambio de información estructurada

Protocolo HTTP: HyperText Transfer Protocol

Es el protocolo de la capa de aplicación que hace posible la existencia de la World Wide Web (WWW).

Posee dos características de diseño arquitectónico fundamentales:

- **Basado en Texto Plano:** Los mensajes son fácilmente legibles por humanos y herramientas de depuración.
- **Sin Estado (Stateless):** El servidor no guarda ninguna memoria de las peticiones anteriores de forma nativa. Cada petición se procesa de forma aislada y limpia.

Métodos o Verbos HTTP Semánticos

Indican explícitamente la naturaleza de la operación que el cliente desea realizar en el servidor:

GET

Solicitar o recuperar un recurso del servidor sin alterarlo.

POST

Enviar datos nuevos al servidor para crear un registro nuevo.

DELETE

Instruir la eliminación física o lógica de un recurso específico.



Beneficios de Ingeniería de las Arquitecturas N-Tier

Por qué las grandes corporaciones prohíben el desarrollo de aplicaciones acopladas

Ventaja 1: Escalabilidad Independiente

En épocas de alta demanda (ej. Periodo de asignación de cursos en la USAC), la capa que más sufre saturación es el Nivel de Aplicación debido al procesamiento de algoritmos.

Aumento de Recursos Quirúrgico: Gracias al desacoplamiento físico de la arquitectura N-Tier, podemos clonar y añadir 5 servidores más a la capa de negocio sin necesidad de gastar dinero en comprar más almacenamiento o modificar las computadoras de los clientes.

Ventaja 2: Mantenimiento y Robustez

Imaginemos que la universidad decide cambiar por completo su base de datos relacional de SQL Server a un motor moderno de código abierto como PostgreSQL.

Cero Impacto Visual: La modificación de los controladores y cadenas de conexión se realiza exclusivamente dentro de las fronteras físicas de la Capa de Datos. La interfaz gráfica en los teléfonos de los estudiantes no sufre un solo cambio de línea ni requiere actualizaciones en las tiendas de apps.



Buenas Prácticas en el Modelado de Redes

Estándares internacionales para garantizar la protección perimetral del software empresarial

Error Fatal: Exponer Puertos de Datos a Internet

Un error crítico de arquitectura muy común cometido por desarrolladores junior es configurar la base de datos con una IP pública expuesta a todo internet para "facilitar" las pruebas locales.

Riesgo Crítico de Seguridad: Hackers globales escanean constantemente rangos de IP buscando puertos de bases de datos abiertos para secuestrar y borrar información mediante ataques automatizados.

 **Buenas Práctica:** Bloquear el acceso externo total a la base de datos, abriendo únicamente comunicación exclusiva de entrada proveniente de la dirección IP del servidor backend.

Valor de la Unidad: Solidaridad



"El primer paso en la evolución de la ética es la solidaridad con los demás seres humanos."

— **Albert Schweitzer**

****Aplicación en Informática:**** Diseñar arquitecturas web eficientes y desacopladas es una expresión de solidaridad técnica. Asegura que los servicios públicos (como inscripciones, salud o banca social) permanezcan en línea y accesibles para toda la población, sin colapsar ante la necesidad colectiva.

Conclusiones de la Sesión

Cliente-Servidor

Modelo asimétrico donde los clientes solicitan recursos bajo demanda y los servidores centralizados los procesan e intersectan.

Modelo 3-Tier

Separación física e independiente de los componentes de Presentación, Lógica de Negocio y Persistencia de Datos.

Protocolo HTTP

La norma de comunicación universal de la web, caracterizada por ser estructurada en texto plano y no poseer estado nativo.

Próximo Paso: Sesión 12

El Patrón de Diseño Modelo-Vista-Controlador (MVC)

En la próxima clase traduciremos estos conceptos físicos a código y patrones de desarrollo de software. Estudiaremos a fondo la ****Sesión 12: Arquitectura y componentes del patrón MVC****, comprendiendo el ciclo de enrutamiento y la interacción de componentes en aplicaciones interactivas.

¿Preguntas sobre Arquitecturas Web o Modelos de Despliegue?

Es momento de abrir el debate y analizar las redes que conectan nuestro código.



Foros de Discusión UEDI



Contacto Técnico: 3021894750101@ingenieria.usac.edu.gt